

User Manual

for S32K1_S32M24X OCOTP Driver

Document Number: UM2OCOTPASRR21-11 Rev0000R3.0.0 Rev. 1.0

1 Revision History	2
2 Introduction	3
2.1 Supported Derivatives	3
2.2 Overview	3
2.3 About This Manual	4
2.4 Acronyms and Definitions	5
2.5 Reference List	5
3 Driver	6
3.1 Requirements	6
3.2 Driver Design Summary	6
3.3 Hardware Resources	7
3.4 Deviations from Requirements	7
3.5 Driver Limitations	7
3.6 Driver usage and configuration tips	7
3.6.1 Fuse Map Table	7
3.6.2 Ocotp Timeout Supervision	8
3.6.3 Configuration	8
3.7 Runtime errors	8
3.8 Symbolic Names Disclaimer	9
3.9 Usage in non AUTOSAR context	9
4 Configuration Parameters	11
4.1 Ocotp	12
4.1.1 Module Ocotp	13
4.1.2 Container OcotpGeneral	13
4.1.3 Parameter OcotpDevErrorDetect	13
4.1.4 Parameter OcotpIpDevErrorDetect	14
4.1.5 Parameter OcotpEnableUserModeSupport	14
4.1.6 Parameter OcotpEnableMultiPartition	14
4.1.7 Parameter OcotpReadEFuseApi	15
4.1.8 Parameter OcotpGetStatusApi	15
4.1.9 Parameter OcotpTimeoutMethod	16
4.1.10 Parameter OcotpWriteTimeout	16
4.1.11 Parameter OcotpReadTimeout	17
4.1.12 Container OcotpConfigList	17
4.1.13 Container OcotpHwUnit	17
4.1.14 Parameter OcotpHardwareIndex	18
4.1.15 Parameter OcotpInstance	18
4.1.16 Reference OcotpEcucPartitionRef	19
4.1.17 Container OcotpChannelList	19
4.1.18 Container OcotpChannel	19
4.1.19 Parameter OcotpChannelNumber	20
4.1.20 Parameter OcotpWord	20
4.1.21 Reference OcotpHwRef	21
4.1.22 Container CommonPublishedInformation	21
4.1.23 Parameter ArReleaseMajorVersion	21
4.1.24 Parameter ArReleaseMinorVersion	22
4.1.25 Parameter ArReleaseRevisionVersion	22
4.1.26 Parameter ModuleId	22
4.1.27 Parameter SwMajorVersion	23
4.1.28 Parameter SwMinorVersion	23
4.1.29 Parameter SwPatchVersion	24

4.1.30 Parameter VendorApiInfix	24
4.1.31 Parameter VendorId	25
4.2 MEM_OTP_IP	25
4.2.1 Module Mem_Otp_Ip	26
5 Module Index	27
5.1 Software Specification	27
6 Module Documentation	28
6.1 OCOTP	28
6.1.1 Detailed Description	28
6.1.2 Data Structure Documentation	29
6.1.2.1 struct Ocotp_ChannelConfigType	29
6.1.2.2 struct Ocotp_ConfigType	29
6.1.2.3 struct Ocotp_StatusType	30
6.1.3 Macro Definition Documentation	30
6.1.3.1 OCOTP_MODULE_ID	30
6.1.3.2 OCOTP_INSTANCE_ID	30
6.1.3.3 OCOTP_E_PARAM_CONFIG	30
6.1.3.4 OCOTP_E_PARAM_POINTER	30
6.1.3.5 OCOTP_E_PARAM_CHANNEL	31
6.1.3.6 OCOTP_E_PARAM_INSTANCE	31
6.1.3.7 OCOTP_E_UNINIT	31
6.1.3.8 OCOTP_E_ALREADY_INITIALIZED	31
6.1.3.9 OCOTP_E_BUSY	31
6.1.3.10 OCOTP_E_TIMEOUT	31
6.1.3.11 OCOTP_E_READ_FAILED	32
6.1.3.12 OCOTP_INIT_ID	32
6.1.3.13 OCOTP_DEINIT_ID	32
6.1.3.14 OCOTP_READ_EFUSE_ID	32
6.1.3.15 OCOTP_GET_STATUS_ID	32
6.1.4 Types Reference	32
6.1.4.1 Ocotp_AddressType	32
6.1.4.2 Ocotp_ChannelType	33
6.1.5 Enum Reference	33
6.1.5.1 Ocotp_HardwareStatusType	33
6.1.6 Function Reference	33
6.1.6.1 Ocotp_Init()	33
6.1.6.2 Ocotp_Deinit()	33
6.1.6.3 Ocotp_ReadEFuse()	34
6.1.6.4 Ocotp_GetStatus()	34
6.2 MEM_OTP_IP	36
6.2.1 Detailed Description	36
6.2.2 Data Structure Documentation	36
6.2.2.1 struct Mem_Otp_Ip_ConfigType	36
6.2.3 Types Reference	36
6.2.3.1 Ocotp_InstanceType	36
6.2.4 Enum Reference	37
6.2.4.1 Mem_Otp_Ip_StatusType	37
6.2.5 Function Reference	37
6.2.5.1 Mem_Otp_Ip_Init()	37
6.2.5.2 Mem_Otp_Ip_DeInit()	37
6.2.5.3 Mem_Otp_Ip_ReadOTPMem()	38
6.2.5.4 Mem_Otp_Ip_GetBusyState()	38



Chapter 1

Revision History

Revision	Date	Author	Description
1.0	07.03.2025	NXP RTD Team	S32K1_S32M24X Real-Time Drivers AUTOSAR R21-11 Version 3.0.0

Chapter 2

Introduction

- [Supported Derivatives](#)
- [Overview](#)
- [About This Manual](#)
- [Acronyms and Definitions](#)
- [Reference List](#)

This User Manual describes for One Time Programmable Memory (MEM OTP) Driver for S32M24X. MEM OTP driver configuration parameters and deviations from the specification are described in Driver chapter of this document.

2.1 Supported Derivatives

The software described in this document is intended to be used with the following microcontroller devices of NXP Semiconductors:

- s32m241_lqfp64
- s32m242_lqfp64
- s32m243_lqfp64
- s32m244_lqfp64

All of the above microcontroller devices are collectively named as S32K1_S32M24X. Note: MWCT part numbers contain NXP confidential IP for Qi Wireless Power

2.2 Overview

AUTOSAR (AUTomotive Open System ARchitecture) is an industry partnership working to establish standards for software interfaces and software modules for automobile electronic control systems.

AUTOSAR:

- paves the way for innovative electronic systems that further improve performance, safety and environmental friendliness.
- is a strong global partnership that creates one common standard: "Cooperate on standards, compete on implementation".

- is a key enabling technology to manage the growing electrics/electronics complexity. It aims to be prepared for the upcoming technologies and to improve cost-efficiency without making any compromise with respect to quality.
- facilitates the exchange and update of software and hardware over the service life of the vehicle.

2.3 About This Manual

This Technical Reference employs the following typographical conventions:

- **Boldface** style: Used for important terms, notes and warnings.
- *Italic* style: Used for code snippets in the text. Note that C language modifiers such "const" or "volatile" are sometimes omitted to improve readability of the presented code.

Notes and warnings are shown as below:

Note

This is a note.

Warning

This is a warning

2.4 Acronyms and Definitions

Term	Definition
API	Application Programming Interface
AUTOSAR	Automotive Open System Architecture
DET	Development Error Tracer
ECU	Electronic Control Unit
MCU	Micro Controller Unit
OS	Operating System
MSB	Most Significant Bit
N/A	Not Applicable
OCOTP	On-Chip One Time Programmable
MEM OTP	One time programmable memory
OTPC	One-time-programmable memory controller
HW	Hardware

2.5 Reference List

#	Title	Version
1	S32K1xx Reference Manual	Rev.14.1, 01/2024
2	S32M24x Reference Manual	Rev. 5, 12/2024
3	S32K1xx Data Sheet	Rev. 14, 08/2021
4	S32M2xx Data Sheet	Rev. 7, 12/2024
5	S32K116 Errata	Mask Set Errata for Mask 0N96V Rev. 17/JAN/2024, 1/2024
6	S32K118 Errata	Mask Set Errata for Mask 0N97V Rev. 15/JAN/2024, 1/2024
7	S32K142 Errata	Mask Set Errata for Mask 0N33V Rev. 15/JAN/2024, 1/2024
8	S32K144 Errata	Mask Set Errata for Mask 0N57U Rev. 15/JAN/2024, 1/2024
9	S32K144W Errata	Mask Set Errata for Mask 0P64A Rev. 03/JUL/2024, 7/2024
10	S32K146 Errata	Mask Set Errata for Mask 0N73V Rev. 15/JAN/2024, 1/2024
11	S32K148 Errata	Mask Set Errata for Mask 0N20V Rev. 15/JAN/2024, 1/2024
12	S32M242 Errata	Mask Set Errata for Mask N33V+P69K Rev.1, 8/2024
13	S32M244 Errata	Mask Set Errata for Mask P64A+P69K Rev.1, 8/2024

Chapter 3

Driver

- [Requirements](#)
- [Driver Design Summary](#)
- [Hardware Resources](#)
- [Deviations from Requirements](#)
- [Driver Limitations](#)
- [Driver usage and configuration tips](#)
- [Runtime errors](#)
- [Symbolic Names Disclaimer](#)
- [Usage in non AUTOSAR context](#)

3.1 Requirements

Deviations from requirements are described in [Deviations from Requirements](#) chapter of this document.

3.2 Driver Design Summary

MEM_OTP contains a one-time programmable fuse block (OTP), an associated controller OTPC, and a set of mirror registers. It supports the execution of commands to manage the OTP memory. The OTPC has 3 functions:

- At boot time, it reads the contents of the OTP memory(eFuse) into the mirror registers.
- It executes commands in response to the OTPC registers
- It performs tests on the OTP memory

MEM_OTP driver is designed to provide APIs to implement reading OTP content. That is information data, that are stored from address 64 to 75 ("ECID") and address 96 to 99 ("Temperature sensor"), can be read by user.

The user performs reading eFuse bytes by configuring the channels. Each channel includes:

- Channel name, channel index that is used to call parameters for the api in the MEM_OTP driver.
- An eFuse byte, includes information:
 - The name of eFuse.
 - The address of eFuse.

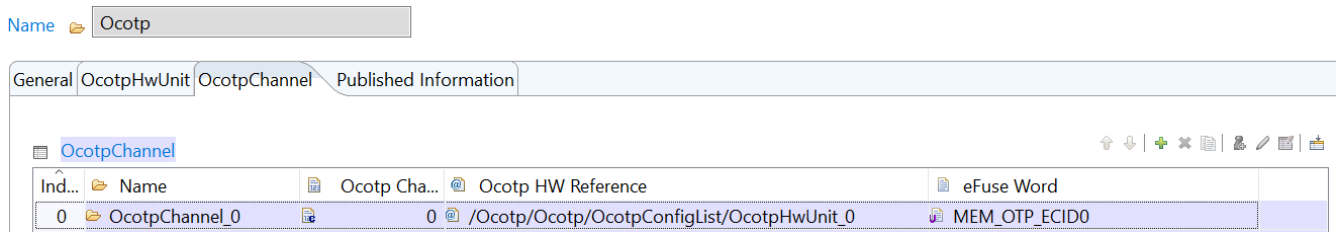


Figure 3.1 Channel configuration

3.3 Hardware Resources

OTP memory can be read is information data that are stored from address 64 to 75 ("ECID") and address 96 to 99 ("Temperature sensor").

3.4 Deviations from Requirements

The driver deviates from the OCOTP Driver software specification in someplaces. There are also some additional requirements (on top of requirements detailed in OCOTP Driver software specification) which need to be satisfied for correct operation.

- Deviations Status Column Description

Term	Definition
N/S	Not In Scope
N/F	Not Fully Implemented
N/I	Not Implemented

None.

3.5 Driver Limitations

- Before reading OTP memory, need to write value 0x455C and 0xE80C into MODE_CONTROL register of AEC region to activate low level access to OTP. But this register is not showed in Reference Manual.
- In Mem_Otp_ReadOTPMem function, there are some registers that not be showed in Reference Manual, such as: MREF_TEST (address 0x138), MR_TST(0x13A), ADVCFG0 (0x12C), ADVCFG3 (0x132).

3.6 Driver usage and configuration tips

- [Fuse Map Table](#)
- [Ocotp Timeout Supervision](#)
- [Configuration](#)

3.6.1 Fuse Map Table

FuseMap is divided into 16 registers, each register is 8 bits. by user.

3.6.2 Ocotp Timeout Supervision

Ocotp will be in a busy state if it is in processes:

- Read from eFuse

Therefore, some of the following APIs need to be configured Timeout for OCOTP to complete the process.

- [Ocotp_ReadEFuse\(\)](#);

User can refer to the configuration for Timeout as shown below.

Ocotp Timeout Method

OSIF_COUNTER_DUMMY

Ocotp Read Timeout (0 -> 4294967295)

100

Figure 3.2 Configuration for timeout

3.6.3 Configuration

A channel will contain some independent information:

- General information:
 - Ocotp Channel Number
 - Ocotp HW Reference
- An eFuse Word information:
 - eFuse Word : the eFuse byte address and corresponding Mirror Register address

OcotpChannel

@ ↑ ↗

Name OcotpChannel_0

Ocotp Channel

Ocotp Channel Number 0

Ocotp HW Reference /Ocotp/Ocotp/OcotpConfigList/OcotpHwUnit_0

eFuse Word MEM_OTP_ECID0

Figure 3.3 Channel configuration information

3.7 Runtime errors

- The driver supports runtime generation of the errors listed in the Table:

Function	Error code	Condition triggering the error
Ocotp_ReadEFuse()	OCOTP_E_TIMEOUT	(Ocotp Timeout Supervision Enabled == ON) and Ocotp is still busy when the timeout is over.

- The driver supports Transient faults generation of the errors listed in the Table.

Function	Error Code	Condition triggering the error
<code>Ocotp_ReadEFuse()</code>	OCOTP_E_READ_FAILED	Read operation failed.

3.8 Symbolic Names Disclaimer

All containers having symbolicNameValue set to TRUE in the AUTOSAR schema will generate defines like: `#define <Mip>Conf_<Container_ShortName>_<Container_ID>`

For this reason it is forbidden to duplicate the names of such containers across the RTD configurations or to use names that may trigger other compile issues (e.g. match existing `#ifdefs` arguments).

3.9 Usage in non AUTOSAR context

For the non-ASR environments, both High-Level and IP layer interfaces are exposed and can be used by software units above. These may consist of either higher level NXP SW deliverables, like stacks and middleware, or the application code itself.

The goal of the product is to be versatile, easy to use and integrate with other SW environments, offering both standardized and optimized low-level interfaces.

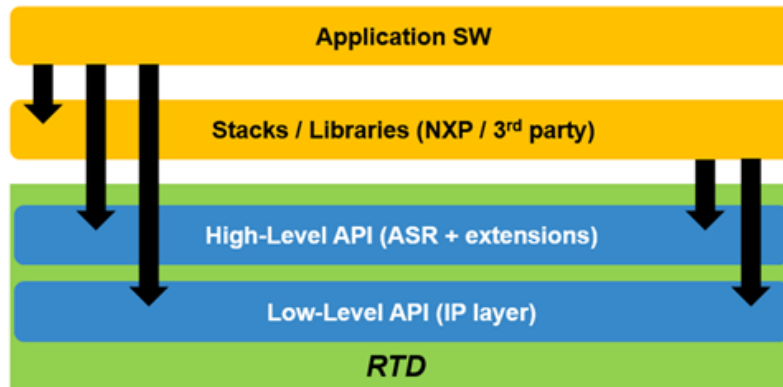


Figure 3.4 RTD Usage in non-ASR Context

In the non-AUTOSAR environment, there are no specific ECU states defined, but an external assumption for the application is that the correct sequence of initialization takes place prior to calling any runtime APIs.

The constraints considered for defining the APIs in the non-AUTOSAR environment are:

- Hardware resources used by the drivers must be properly initialized prior to runtime usage by calling the initialization function with configuration parameters generated by the configuration tools
- All dependencies must be initialized prior to the usage of a driver that depends on them; this translates to a general initialization sequence applicable for all applications, as follows:
 1. Initialization of the device clock tree so other periphery can be initialized
 2. Configuration of the pinout so peripherals with external connections can communicate with the outside world
 3. Configuration of the common resources (e.g. DMA channels, memory regions, access rights, etc) prior to the usage of any runtime functionality that has to benefit from this overall SoC configuration

4. Initialization of specific drivers
5. Runtime functionality
6. Deinitialization (inverse initialization order)

High-level and IP layer functionalities cannot be used simultaneously over the same hardware resource. The granularity of the resource must be defined specifically for each driver, but as a general rule the default granularity considered is the peripheral instance (e.g. Gpt high level APIs cannot be called interchangeably with Pit_Ip low level APIs over the same PIT hardware instance).

The diagram below shows a generic, high-level sequence of calls to RTD interfaces in the non-AUTOSAR scenario:

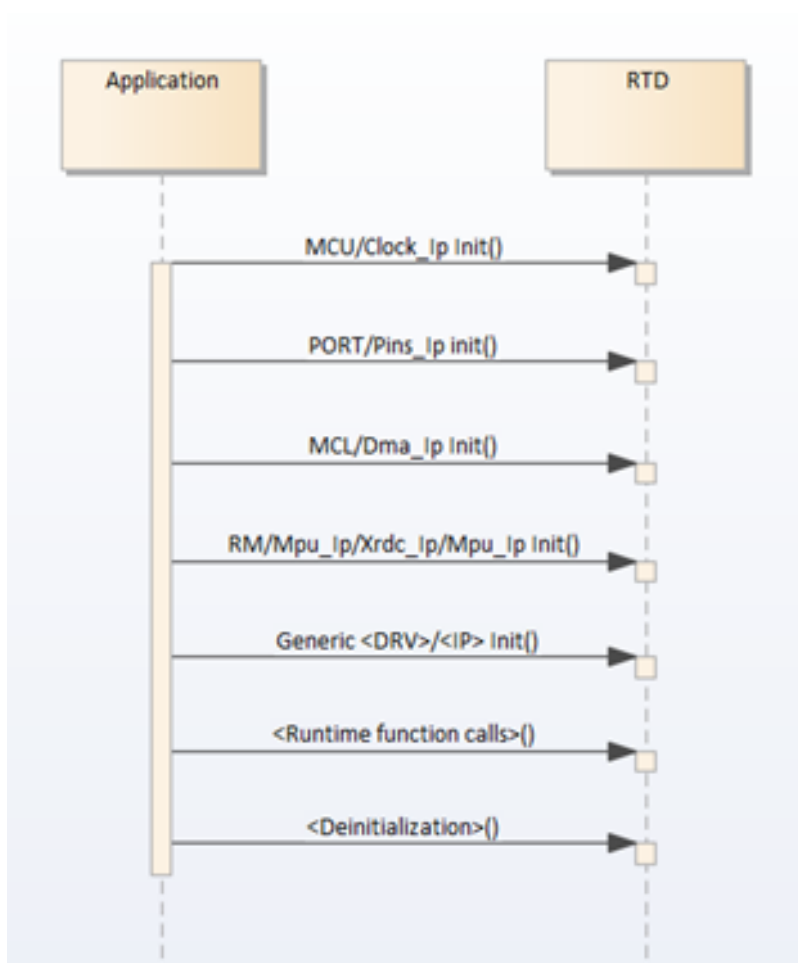


Figure 3.5 Non-AUTOSAR Sequence diagram

Chapter 4

Configuration Parameters

RTD provides configuration schema for the high level driver in EB Tresos proprietary format (**.xdm**), S32CT proprietary format (**.component**), but also in a standardized AUTOSAR format (**.epd**). For the low level driver configuration only the S32CT proprietary format (**.component**) is supported. The configuration data of the project can be saved in EB Tresos proprietary format (**.xdm**) and AUTOSAR standardized format (**.epc**) from EB Tresos and in S32CT proprietary format (**.mex**) and AUTOSAR standardized format (**.epc**) from S32 Design Studio. RTD provides templates for parsing the configuration data and generating configuration structures from it in XPath format, to be used with the EB Tresos generator and in JS format, to be used with the S32CT generator. Since the generation process is not standardized by AUTOSAR, only the two mentioned tools can be used with the provided templates. Using the same input configuration data, the same configuration structures will be obtained using any of the generators.

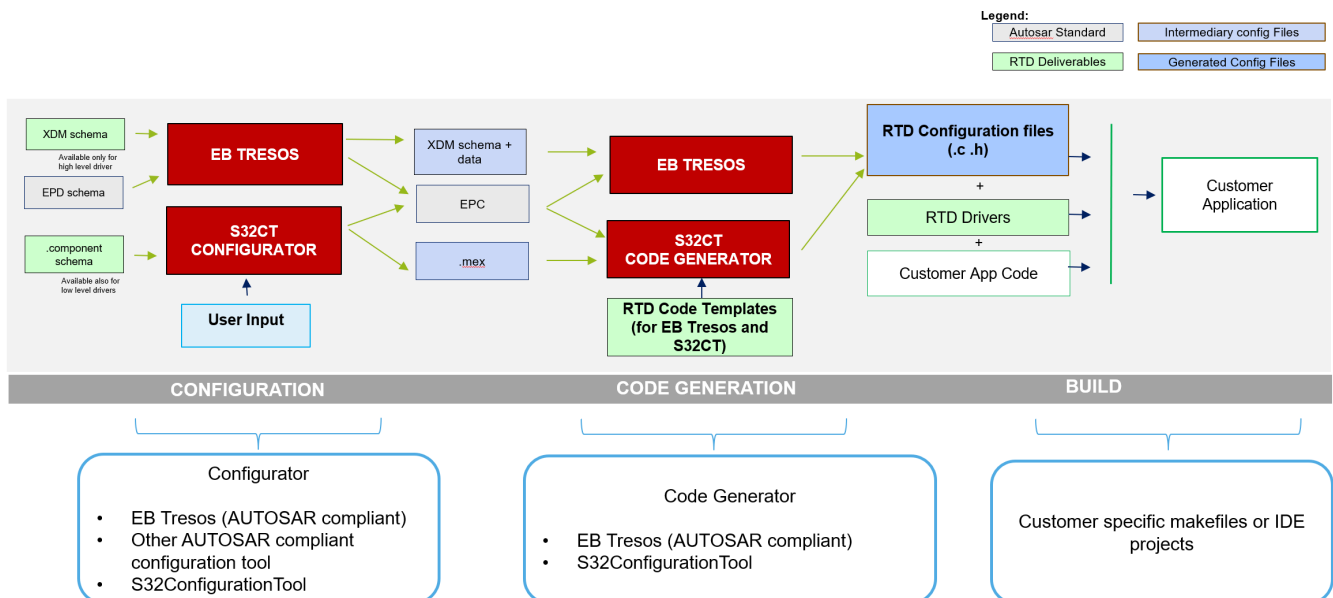


Figure 4.1 Configuration generation flow

- [Ocotp](#)
- [MEM_OTP_IP](#)

4.1 Ocotp

- Module [Ocotp](#)
 - Container [OcotpGeneral](#)
 - * Parameter [OcotpDevErrorDetect](#)
 - * Parameter [OcotpIpDevErrorDetect](#)
 - * Parameter [OcotpEnableUserModeSupport](#)
 - * Parameter [OcotpEnableMultiPartition](#)
 - * Parameter [OcotpReadEFuseApi](#)
 - * Parameter [OcotpGetStatusApi](#)
 - * Parameter [OcotpTimeoutMethod](#)
 - * Parameter [OcotpWriteTimeout](#)
 - * Parameter [OcotpReadTimeout](#)
 - Container [OcotpConfigList](#)
 - * Container [OcotpHwUnit](#)
 - Parameter [OcotpHardwareIndex](#)
 - Parameter [OcotpInstance](#)
 - Reference [OcotpEcucPartitionRef](#)
 - Container [OcotpChannelList](#)
 - * Container [OcotpChannel](#)
 - Parameter [OcotpChannelNumber](#)
 - Parameter [OcotpWord](#)
 - Reference [OcotpHwRef](#)
 - Container [CommonPublishedInformation](#)
 - * Parameter [ArReleaseMajorVersion](#)
 - * Parameter [ArReleaseMinorVersion](#)
 - * Parameter [ArReleaseRevisionVersion](#)
 - * Parameter [ModuleId](#)
 - * Parameter [SwMajorVersion](#)
 - * Parameter [SwMinorVersion](#)
 - * Parameter [SwPatchVersion](#)
 - * Parameter [VendorApiInfix](#)
 - * Parameter [VendorId](#)

4.1.1 Module Ocotp

Configuration of the Ocotp module.

Included containers:

- [OcotpGeneral](#)
- [OcotpConfigList](#)
- [OcotpChannelList](#)
- [CommonPublishedInformation](#)

Property	Value
type	ECUC-MODULE-DEF
lowerMultiplicity	1
upperMultiplicity	Infinite
postBuildVariantSupport	true
supportedConfigVariants	VARIANT-POST-BUILD, VARIANT-PRE-COMPILE

4.1.2 Container OcotpGeneral

Container for general parameters of the ocotp driver. These parameters are always pre-compile.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.1.3 Parameter OcotpDevErrorDetect

Pre-processor switch to enable and disable development error detection.

true: Development error detection enabled.

false: Development error detection disabled.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false

Configuration Parameters

Property	Value
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	true

4.1.4 Parameter OcotpIpDevErrorDetect

Pre-processor switch to enable and disable development error detection for IP layer.

true: Ocotp error detect for IP layer is enabled.

false: Ocotp error detect for IP layer is disabled.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	true

4.1.5 Parameter OcotpEnableUserModeSupport

Ocotp module can run in user mode without any specific measures. The parameter is not used in the Ocotp implementation.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	false

4.1.6 Parameter OcotpEnableMultiPartition

When this parameter is enabled, the OCOTP module will adapt to run with MultiPartition.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	false

4.1.7 Parameter OcotpReadEFuseApi

Compile switch to enable and disable the *Ocotp_ReadEFuse* function.

true: API supported / function provided.

false: API not supported / function not provided

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	true

4.1.8 Parameter OcotpGetStatusApi

Compile switch to enable and disable the *Ocotp_GetStatus* function.

true: API supported / function provided.

false: API not supported / function not provided

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1

Property	Value
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	true

4.1.9 Parameter OcotpTimeoutMethod

OcotpTimeoutMethod

Configures the timeout method.

Based on this selection a certain timeout method from OsIf will be used in the driver.

Note: If OSIF_COUNTER_SYSTEM or OSIF_COUNTER_CUSTOM are selected make sure the corresponding timer is enabled in OsIf General configuration.

Note: Implementation Specific Parameter.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
default Value	OSIF_COUNTER_DUMMY
literals	['OSIF_COUNTER_DUMMY', 'OSIF_COUNTER_SYSTEM', 'OSIF_COUNTER_CUSTOM']

4.1.10 Parameter OcotpWriteTimeout

Timeout value for write eFuse operation.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

Property	Value
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	1
max	4294967295
min	0

4.1.11 Parameter OcotpReadTimeout

Timeout value for read eFuse operation.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	1
max	4294967295
min	0

4.1.12 Container OcotpConfigList

Container for runtime configuration parameters of the Ocotp driver.

Implementation Type: Ocotp_ConfigType.

Included subcontainers:

- [OcotpHwUnit](#)

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.1.13 Container OcotpHwUnit

Ocotp Hardware Unit.

Configuration Parameters

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	0
upperMultiplicity	Infinite
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD

4.1.14 Parameter OcotpHardwareIndex

Index of current hardware unit.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	true
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	0
max	255
min	0

4.1.15 Parameter OcotpInstance

Instance of this word channel

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	0

Property	Value
max	0
min	0

4.1.16 Reference OcotpEcucPartitionRef

Maps the OCOTP driver to zero or multiple ECUC partitions to make the driver API available in the according partition.

Property	Value
type	ECUC-REFERENCE-DEF
origin	NXP
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	true
multiplicityConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
requiresSymbolicNameValue	False
destination	/AUTOSAR/EcucDefs/EcuC/EcucPartitionCollection/EcucPartition

4.1.17 Container OcotpChannelList

List of Otp channel.

Included subcontainers:

- [OcotpChannel](#)

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.1.18 Container OcotpChannel

Ocotp Channel Configuration.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1

Configuration Parameters

Property	Value
upperMultiplicity	Infinite
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD

4.1.19 Parameter OcotpChannelNumber

Index of current channel.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	true
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	0
max	255
min	0

4.1.20 Parameter OcotpWord

Ocotp Fuse Word.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	MEM_OTP_ECID0
literals	['MEM_OTP_ECID0', 'MEM_OTP_ECID1', 'MEM_OTP_ECID2', 'MEM_OTP_ECID3', 'MEM_OTP_ECID4', 'MEM_OTP_ECID5', 'MEM_OTP_ECID6', 'MEM_OTP_ECID7', 'MEM_OTP_ECID8', 'MEM_OTP_ECID9', 'MEM_OTP_ECID10', 'MEM_OTP_ECID11', 'MEM_OTP_TEMP0', 'MEM_OTP_TEMP1', 'MEM_OTP_TEMP2', 'MEM_OTP_TEMP3']

4.1.21 Reference OcotpHwRef

Select Ocotp Hardware Reference

Property	Value
type	ECUC-REFERENCE-DEF
origin	NXP
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
requiresSymbolicNameValue	False
destination	/TS_T40D2M30I0R0/Ocotp/OcotpConfigList/OcotpHwUnit

4.1.22 Container CommonPublishedInformation

Common container, aggregated by all modules. It contains published information about vendor and versions.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.1.23 Parameter ArReleaseMajorVersion

Vendor specific: Major version number of AUTOSAR specification on which the appropriate implementation is based on.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION

Configuration Parameters

Property	Value
defaultValue	4
max	4
min	4

4.1.24 Parameter ArReleaseMinorVersion

Vendor specific: Minor version number of AUTOSAR specification on which the appropriate implementation is based on.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	7
max	7
min	7

4.1.25 Parameter ArReleaseRevisionVersion

Vendor specific: Patch version number of AUTOSAR specification on which the appropriate implementation is based on.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	0
max	0
min	0

4.1.26 Parameter ModuleId

Vendor specific: Module ID of this module.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	255
max	255
min	255

4.1.27 Parameter SwMajorVersion

Major version number of the vendor specific implementation of the module. The numbering is vendor specific.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	3
max	3
min	3

4.1.28 Parameter SwMinorVersion

Minor version number of the vendor specific implementation of the module. The numbering is vendor specific.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A

Configuration Parameters

Property	Value
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	0
max	0
min	0

4.1.29 Parameter SwPatchVersion

Patch level version number of the vendor specific implementation of the module. The numbering is vendor specific.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	0
max	0
min	0

4.1.30 Parameter VendorApiInfix

In driver modules which can be instantiated several times on a single ECU, BSW00347 requires that the name of APIs is extended by the VendorId and a vendor specific name.

This parameter is used to specify the vendor specific name. In total, the implementation specific name is generated as follows:

`<ModuleName>_>VendorId>_<VendorApiInfix>.`

E.g. assuming that the VendorId of the implementor is 123 and the implementer chose a VendorApiInfix of "v11r456" a api name Can_Write defined in the SWS will translate to Can_123_v11r456Write.

This parameter is mandatory for all modules with upper multiplicity > 1. It shall not be used for modules with upper multiplicity =1.

Property	Value
type	ECUC-STRING-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	false

Property	Value
multiplicityConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	

4.1.31 Parameter VendorId

Vendor ID of the dedicated implementation of this module according to the AUTOSAR vendor list.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	43
max	43
min	43

4.2 MEM_OTP_IP

The low level driver component can only be used with S32CT Configurator in S32 Design Studio.

- Module [Mem_Otp_Ip](#)
 - Container [OcotpGeneral](#)
 - * Parameter [OcotpIpDevErrorDetect](#)
 - * Parameter [OcotpEnableUserModeSupport](#)
 - * Parameter [OcotpReadEFuseApi](#)
 - * Parameter [OcotpTimeoutMethod](#)
 - * Parameter [OcotpWriteTimeout](#)
 - * Parameter [OcotpReadTimeout](#)
 - Container [OcotpConfigList](#)
 - * Container [OcotpHwUnit](#)
 - Parameter [OcotpInstance](#)

4.2.1 Module Mem_Otp_Ip

Ocotp config set

Included containers:

- [OcotpGeneral](#)
- [OcotpConfigList](#)

Property	Value
type	MODULE-DEF
lowerMultiplicity	1
upperMultiplicity	Infinite
postBuildVariantSupport	true
supportedConfigVariants	VARIANT-POST-BUILD;VARIANT-PRE-COMPILE



Chapter 5

Module Index

5.1 Software Specification

Here is a list of all modules:

OCOTP	28
MEM_OTP_IP	36

Chapter 6

Module Documentation

6.1 OCOTP

6.1.1 Detailed Description

Data Structures

- struct [Ocotp_ChannelConfigType](#)
Ocotp channel configuration type. [More...](#)
- struct [Ocotp_ConfigType](#)
Ocotp configuration type. [More...](#)
- struct [Ocotp_StatusType](#)
Ocotp status type. [More...](#)

Macros

- [#define OCOTP_MODULE_ID](#)
ID of module and instance.
- [#define OCOTP_INSTANCE_ID](#)
- [#define OCOTP_E_PARAM_CONFIG](#)
Development error codes (passed to DET).
- [#define OCOTP_E_PARAM_POINTER](#)
- [#define OCOTP_E_PARAM_CHANNEL](#)
- [#define OCOTP_E_PARAM_INSTANCE](#)
- [#define OCOTP_E_UNINIT](#)
- [#define OCOTP_E_ALREADY_INITIALIZED](#)
- [#define OCOTP_E_BUSY](#)
- [#define OCOTP_E_TIMEOUT](#)
Runtime error codes (passed to DET).
- [#define OCOTP_E_READ_FAILED](#)
Transient Faults codes (passed to DET).
- [#define OCOTP_INIT_ID](#)
All service IDs (passed to DET).
- [#define OCOTP_DEINIT_ID](#)

- `#define OCOTP_READ_EFUSE_ID`
- `#define OCOTP_GET_STATUS_ID`

Types Reference

- `typedef uint32 Ocotp_AddressType`
Ocotp address type.
- `typedef uint8 Ocotp_ChannelType`
Ocotp channel type.

Enum Reference

- `enum Ocotp_HardwareStatusType`
Status of Ocotp hardware.

Function Reference

- `void Ocotp_Init (const Ocotp_ConfigType *const ConfigPtr)`
The function initializes Ocotp module.
- `void Ocotp_Deinit (void)`
The function de-initializes Ocotp module.
- `Std_ReturnType Ocotp_ReadEFuse (Ocotp_ChannelType eFuseChannel, uint32 *pData)`
The function reads data from efuse word.
- `Std_ReturnType Ocotp_GetStatus (uint8 HwIndex, Ocotp_StatusType *pStatus)`
The function gets status of Ocotp module.

6.1.2 Data Structure Documentation

6.1.2.1 struct Ocotp_ChannelConfigType

Ocotp channel configuration type.

A structure which is used to contain the parameters of channel used. Such as address of eFuse. Index of shadow register.

Definition at line 152 of file Ocotp_Types.h.

Data Fields

<code>uint32</code>	<code>Ocotp_MirrorAddress</code>	Address of Mirror Register
<code>uint8</code>	<code>Ocotp_ChannelNumber</code>	Channel number
<code>Ocotp_InstanceType</code>	<code>Ocotp_Instance</code>	Instance Index

6.1.2.2 struct Ocotp_ConfigType

Ocotp configuration type.

A structure which is used to contain the hardware configuration and the configuration of channels used.

Definition at line 165 of file Ocotp_Types.h.

Data Fields

const Mem_Otp_Ip_ConfigType *	pHwConfig	User configuration structure
const Ocotp_ChannelConfigType (*	pChanelConfig)[]	Ocotp channel configuration type

6.1.2.3 struct Ocotp_StatusType

Ocotp status type.

A structure which is used to contain the status of hardware (Busy or idle or errors occurred) and status of repair error flag(If FBXC exists).

Definition at line 180 of file Ocotp_Types.h.

6.1.3 Macro Definition Documentation

6.1.3.1 OCOTP_MODULE_ID

```
#define OCOTP_MODULE_ID
```

ID of module and instance.

Ocotp module ID

Definition at line 146 of file CDD_Ocotp.h.

6.1.3.2 OCOTP_INSTANCE_ID

```
#define OCOTP_INSTANCE_ID
```

Ocotp instance ID

Definition at line 147 of file CDD_Ocotp.h.

6.1.3.3 OCOTP_E_PARAM_CONFIG

```
#define OCOTP_E_PARAM_CONFIG
```

Development error codes (passed to DET).

API service called with wrong parameter

Definition at line 154 of file CDD_Ocotp.h.

6.1.3.4 OCOTP_E_PARAM_POINTER

```
#define OCOTP_E_PARAM_POINTER
```

API service called with invalid pointer

Definition at line 155 of file CDD_Ocotp.h.

6.1.3.5 OCOTP_E_PARAM_CHANNEL

```
#define OCOTP_E_PARAM_CHANNEL
```

API service used with an invalid channel identifier or channel was not configured for the functionality of the calling API

Definition at line 156 of file CDD_Ocotp.h.

6.1.3.6 OCOTP_E_PARAM_INSTANCE

```
#define OCOTP_E_PARAM_INSTANCE
```

API service used with an invalid hardware unit identifier or hardware unit was not configured for the functionality of the calling API

Definition at line 157 of file CDD_Ocotp.h.

6.1.3.7 OCOTP_E_UNINIT

```
#define OCOTP_E_UNINIT
```

API service called without module initialization

Definition at line 158 of file CDD_Ocotp.h.

6.1.3.8 OCOTP_E_ALREADY_INITIALIZED

```
#define OCOTP_E_ALREADY_INITIALIZED
```

Ocotp_init service called module initialization

Definition at line 159 of file CDD_Ocotp.h.

6.1.3.9 OCOTP_E_BUSY

```
#define OCOTP_E_BUSY
```

API service called while driver still busy

Definition at line 160 of file CDD_Ocotp.h.

6.1.3.10 OCOTP_E_TIMEOUT

```
#define OCOTP_E_TIMEOUT
```

Runtime error codes (passed to DET).

Timeout exceeded

Definition at line 165 of file CDD_Ocotp.h.

6.1.3.11 OCOTP_E_READ_FAILED

```
#define OCOTP_E_READ_FAILED
```

Transient Faults codes (passed to DET).

Ocotp read failed (HW)

Definition at line 171 of file CDD_Ocotp.h.

6.1.3.12 OCOTP_INIT_ID

```
#define OCOTP_INIT_ID
```

All service IDs (passed to DET).

Service ID of Ocotp_Init function

Definition at line 176 of file CDD_Ocotp.h.

6.1.3.13 OCOTP_DEINIT_ID

```
#define OCOTP_DEINIT_ID
```

Service ID of Ocotp_Deinit function

Definition at line 177 of file CDD_Ocotp.h.

6.1.3.14 OCOTP_READ_EFUSE_ID

```
#define OCOTP_READ_EFUSE_ID
```

Service ID of Ocotp_ReadEFuse function

Definition at line 178 of file CDD_Ocotp.h.

6.1.3.15 OCOTP_GET_STATUS_ID

```
#define OCOTP_GET_STATUS_ID
```

Service ID of Ocotp_GetStatus function

Definition at line 180 of file CDD_Ocotp.h.

6.1.4 Types Reference

6.1.4.1 Ocotp_AddressType

```
typedef uint32 Ocotp_AddressType
```

Ocotp address type.

Address of a register.

Definition at line 137 of file Ocotp_Types.h.

6.1.4.2 Ocotp_ChannelType

```
typedef uint8 Ocotp_ChannelType
```

Ocotp channel type.

An integer which is used to describe the order of channel configured.

Definition at line 144 of file Ocotp_Types.h.

6.1.5 Enum Reference

6.1.5.1 Ocotp_HardwareStatusType

```
enum Ocotp_HardwareStatusType
```

Status of Ocotp hardware.

Enumerator

OCOTP_HARDWARE_BUSY	Status of hardware is busy
OCOTP_HARDWARE_ERROR	Status of hardware is error
OCOTP_HARDWARE_IDLE	Status of hardware is idle

Definition at line 124 of file Ocotp_Types.h.

6.1.6 Function Reference

6.1.6.1 Ocotp_Init()

```
void Ocotp_Init (
    const Ocotp_ConfigType *const ConfigPtr )
```

The function initializes Ocotp module.

The function sets the internal module variables according to given configuration set.

Parameters

in	<i>ConfigPtr</i>	Pointer to Ocotp driver configuration set.
----	------------------	--

Precondition

Ocotp_pConfigPtr must be NULL_PTR.

6.1.6.2 Ocotp_Deinit()

```
void Ocotp_Deinit (
```

```
void )
```

The function de-initializes Ocotp module.

The function sets the Ocotp module's status to un-initialized.

Parameters

in	<i>none.</i>	
----	--------------	--

Precondition

Ocotp_pConfigPtr must not be NULL_PTR.

6.1.6.3 Ocotp_ReadEFuse()

```
Std_ReturnType Ocotp_ReadEFuse (
    Ocotp_ChannelType eFuseChannel,
    uint32 * pData )
```

The function reads data from efuse word.

The function reads the data from the eFuse which was configured by configuration tool.

Parameters

in	<i>eFuseChannel</i>	Channel of eFuse such as 0, 1, 2 which was configured by user.
in	<i>data</i>	pointer points to the data read from eFuse word.

Returns

Std_ReturnType

Return values

<i>E_OK</i>	Read operation is successful.
<i>E_NOT_OK</i>	Read operation failed.

Precondition

Ocotp_pConfigPtr must not be NULL_PTR.

6.1.6.4 Ocotp_GetStatus()

```
Std_ReturnType Ocotp_GetStatus (
    uint8 HwIndex,
    Ocotp_StatusType * pStatus )
```

The function gets status of Ocotp module.

The function gets status of Ocotp module Secure mode is enabled or not. Error occurred or not and Status of hardware.

Parameters

in	<i>HwIndex</i>	Index of hardware configured.
in	<i>pStatus</i>	Pointer points to status structure.

Returns

Std_ReturnType

Precondition

Ocotp_pConfigPtr must not be NULL_PTR.

6.2 MEM_OTP_IP

6.2.1 Detailed Description

Data Structures

- struct [Mem_Otp_Ip_ConfigType](#)
User configuration structure. [More...](#)

Types Reference

- typedef uint8 [Ocotp_InstanceType](#)
Ocotp instance type.

Enum Reference

- enum [Mem_Otp_Ip_StatusType](#)
Ocotp Ip status type.

Function Reference

- void [Mem_Otp_Ip_Init](#) (const [Mem_Otp_Ip_ConfigType](#) *pConfig)
The function initializes Mem_Otp_Ip module.
- void [Mem_Otp_Ip_DeInit](#) (uint32 instance)
The function de-initializes Mem_Otp_Ip module.
- [Mem_Otp_Ip_StatusType](#) [Mem_Otp_Ip_ReadOTPMem](#) (uint32 instance, uint32 address, uint32 *data)
The function reads a single byte of data from OTP memory.
- [Mem_Otp_Ip_StatusType](#) [Mem_Otp_Ip_GetBusyState](#) (uint32 instance)
The function checks busy and error state of hardware.

6.2.2 Data Structure Documentation

6.2.2.1 struct Mem_Otp_Ip_ConfigType

User configuration structure.

Structure contains the configuration parameters which will be used to initialize Ocotp module.

Definition at line 105 of file Mem_Otp_Ip_Types.h.

Data Fields

Ocotp_InstanceType	ocotpInstance	Ocotp instance
------------------------------------	---------------	----------------

6.2.3 Types Reference

6.2.3.1 Ocotp_InstanceType

typedef uint8 [Ocotp_InstanceType](#)

Ocotp instance type.

An integer which describe the order of the instance configured.

Definition at line 99 of file Mem_Otp_Ip_Types.h.

6.2.4 Enum Reference

6.2.4.1 Mem_Otp_Ip_StatusType

enum `Mem_Otp_Ip_StatusType`

Ocotp Ip status type.

Enumerator

<code>STATUS_MEM_OTP_IP_ERROR</code>	Boot Error
<code>STATUS_MEM_OTP_IP_BUSY</code>	Status busy
<code>STATUS_MEM_OTP_IP_SUCCESS</code>	Status success
<code>STATUS_MEM_OTP_IP_INVALID_INPUT_ADDRESS</code>	invalid input parameter
<code>STATUS_MEM_OTP_IP_ERROR_TIMEOUT</code>	timeout error

Definition at line 83 of file Mem_Otp_Ip_Types.h.

6.2.5 Function Reference

6.2.5.1 Mem_Otp_Ip_Init()

```
void Mem_Otp_Ip_Init (
    const Mem_Otp_Ip_ConfigType * pConfig )
```

The function initializes Mem_Otp_Ip module.

The function clear all errors flag and initializes Mem_Otp_Ip module.

Parameters

in	<i>pConfig</i>	pointer points to configuration structure.
----	----------------	--

Precondition

Module has been initialized.

6.2.5.2 Mem_Otp_Ip_DeInit()

```
void Mem_Otp_Ip_DeInit (
    uint32 instance )
```

Module Documentation

The function de-initializes Mem_Otp_Ip module.

The function de-initializes Mem_Otp_Ip module.

Parameters

in	<i>instance</i>	hardware instance.
----	-----------------	--------------------

Precondition

Module has been initialized.

6.2.5.3 Mem_Otp_Ip_ReadOTPMem()

```
Mem_Otp_Ip_StatusType Mem_Otp_Ip_ReadOTPMem (
    uint32 instance,
    uint32 address,
    uint32 * data )
```

The function reads a single byte of data from OTP memory.

The function reads a single byte of data from OTP memory.

Parameters

in	<i>instance</i>	hardware instance.
in	<i>address</i>	address of OTP memory.
out	<i>data</i>	pointer points to data result.

Returns

Mem_Otp_Ip_StatusType

Return values

<i>STATUS_MEM_OTP_IP_SUCCESS</i>	if read operation is successful.
<i>STATUS_MEM_OTP_IP_ERROR</i>	if error flag is set.
<i>STATUS_MEM_OTP_IP_BUSY</i>	hardware is busy.

Precondition

Module has been initialized.

6.2.5.4 Mem_Otp_Ip_GetBusyState()

```
Mem_Otp_Ip_StatusType Mem_Otp_Ip_GetBusyState (
    uint32 instance )
```

The function checks busy and error state of hardware.

The function checks busy and error state of hardware.

Parameters

in	<i>instance</i>	hardware instance.
----	-----------------	--------------------

Returns

Mem_Otp_Ip_StatusType

Return values

<i>STATUS_MEM_OTP_IP_SUCCESS</i>	if hardware is ready.
<i>STATUS_MEM_OTP_IP_ERROR</i>	if error flag is set.
<i>STATUS_MEM_OTP_IP_BUSY</i>	hardware is busy.

Precondition

Module has been initialized.

How to Reach Us:

Home Page:

nxp.com

Web Support:

nxp.com/support

Limited warranty and liability - Information in this document is believed to be accurate and reliable. However, NXP Semiconductors does not give any representations or warranties, expressed or implied, as to the accuracy or completeness of such information and shall have no liability for the consequences of use of such information. NXP Semiconductors takes no responsibility for the content in this document if provided by an information source outside of NXP Semiconductors.

In no event shall NXP Semiconductors be liable for any indirect, incidental, punitive, special or consequential damages (including - without limitation - lost profits, lost savings, business interruption, costs related to the removal or replacement of any products or rework charges) whether or not such damages are based on tort (including negligence), warranty, breach of contract or any other legal theory.

Notwithstanding any damages that customer might incur for any reason whatsoever, NXP Semiconductors' aggregate and cumulative liability towards customer for the products described herein shall be limited in accordance with the Terms and conditions of commercial sale of NXP Semiconductors.

Right to make changes - NXP Semiconductors reserves the right to make changes to information published in this document, including without limitation specifications and product descriptions, at any time and without notice. This document supersedes and replaces all information supplied prior to the publication hereof.

Suitability for use in automotive applications - This NXP product has been qualified for use in automotive applications. If this product is used by customer in the development of, or for incorporation into, products or services (a) used in safety critical applications or (b) in which failure could lead to death, personal injury, or severe physical or environmental damage (such products and services hereinafter referred to as "Critical Applications"), then customer makes the ultimate design decisions regarding its products and is solely responsible for compliance with all legal, regulatory, safety, and security related requirements concerning its products, regardless of any information or support that may be provided by NXP. As such, customer assumes all risk related to use of any products in Critical Applications and NXP and its suppliers shall not be liable for any such use by customer. Accordingly, customer will indemnify and hold NXP harmless from any claims, liabilities, damages and associated costs and expenses (including attorneys' fees) that NXP may incur related to customer's incorporation of any product in a Critical Application.

Applications - Applications that are described herein for any of these products are for illustrative purposes only. NXP Semiconductors makes no representation or warranty that such applications will be suitable for the specified use without further testing or modification.

Customers are responsible for the design and operation of their applications and products using NXP Semiconductors products, and NXP Semiconductors accepts no liability for any assistance with applications or customer product design. It is customer's sole responsibility to determine whether the NXP Semiconductors product is suitable and fit for the customer's applications and products planned, as well as for the planned application and use of customer's third party customer(s). Customers should provide appropriate design and operating safeguards to minimize the risks associated with their applications and products.

NXP Semiconductors does not accept any liability related to any default, damage, costs or problem which is based on any weakness or default in the customer's applications or products, or the application or use by customer's third party customer(s). Customer is responsible for doing all necessary testing for the customer's applications and products using NXP Semiconductors products in order to avoid a default of the applications and the products or of the application or use by customer's third party customer(s). NXP does not accept any liability in this respect.

Terms and conditions of commercial sale - NXP Semiconductors products are sold subject to the general terms and conditions of commercial sale, as published at <http://www.nxp.com/profile/terms>, unless otherwise agreed in a valid written individual agreement. In case an individual agreement is concluded only the terms and conditions of the respective agreement shall apply. NXP Semiconductors hereby expressly objects to applying the customer's general terms and conditions with regard to the purchase of NXP Semiconductors products by customer.

Export control - This document as well as the item(s) described herein may be subject to export control regulations. Export might require a prior authorization from competent authorities.

Translations — A non-English (translated) version of a document, including the legal information in that document, is for reference only. The English version shall prevail in case of any discrepancy between the translated and English versions.

Security — Customer understands that all NXP products may be subject to unidentified vulnerabilities or may support established security standards or specifications with known limitations. Customer is responsible for the design and operation of its applications and products throughout their lifecycles to reduce the effect of these vulnerabilities on customer's applications and products. Customer's responsibility also extends to other open and/or proprietary technologies supported by NXP products for use in customer's applications. NXP accepts no liability for any vulnerability. Customer should regularly check security updates from NXP and follow up appropriately.

Customer shall select products with security features that best meet rules, regulations, and standards of the intended application and make the ultimate design decisions regarding its products and is solely responsible for compliance with all legal, regulatory, and security related requirements concerning its products, regardless of any information or support that may be provided by NXP.

NXP has a Product Security Incident Response Team (PSIRT) (reachable at <mailto:PSIRT@nxp.com>) that manages the investigation, reporting, and solution release to security vulnerabilities of NXP products.

NXP B.V. - NXP B.V. is not an operating company and it does not distribute or sell products.